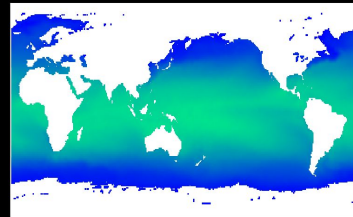
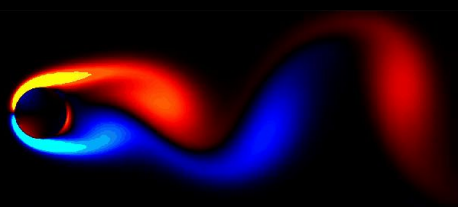


# Open-Source Algorithms for Physics-Informed Data-Driven Modeling in Python



Sara M. Ichinaga and  
Jake Stevens-Haas  
Department of Applied Mathematics  
University of Washington

Steven Brunton  
University of  
Washington

J. Nathan Kutz  
University of  
Washington

Aleksandr Aravkin  
University of  
Washington



APPLIED MATHEMATICS  
UNIVERSITY of WASHINGTON

# Before we get started...

## Presentation overview:

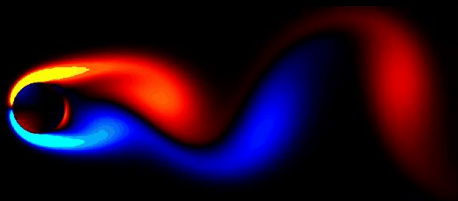
- Introduction to learning dynamics from data
- **Method 1**: dynamic mode decomposition (**DMD**)
  - Fluid dynamics **code demo** with **PyDMD**
- **Method 2**: sparse identification of nonlinear dynamics (**SINDy**)
  - Predator-prey **code demo** with **PySINDy**
- Conclusion and method comparison

Code and slides available at:

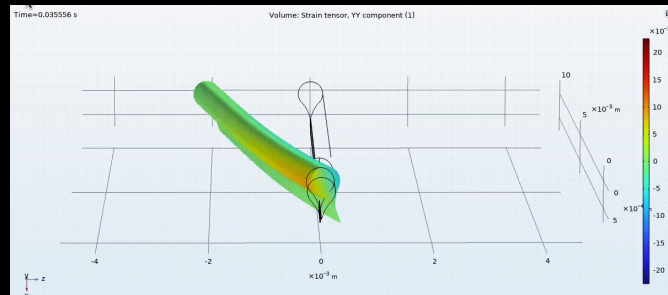
[github.com/sichinaga/python-dynamics-tutorial](https://github.com/sichinaga/python-dynamics-tutorial)

# Introduction and motivation

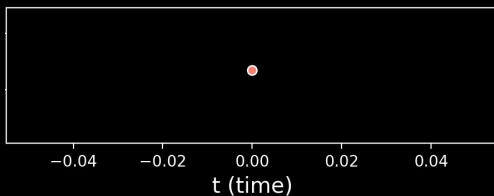
Fluid dynamics



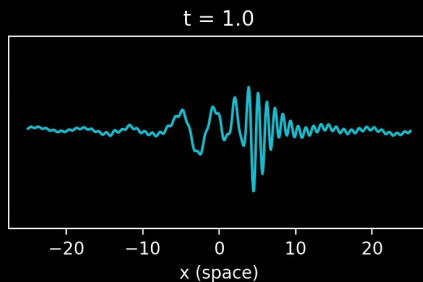
Aeroelasticity



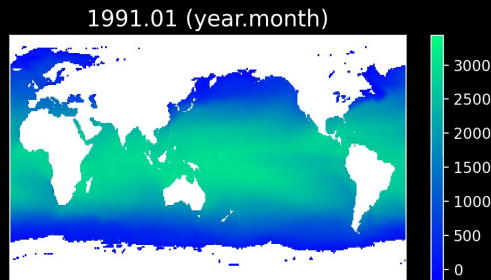
Time-series analysis



Waves and optics

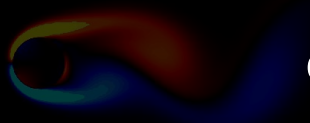


Climate science



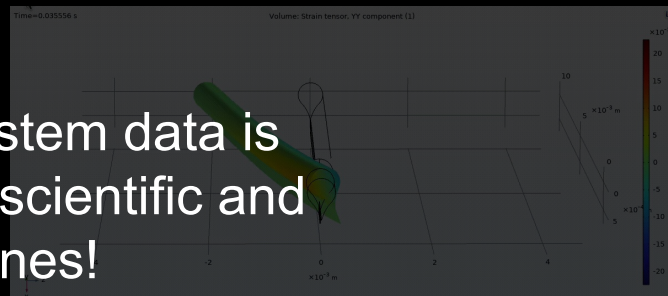
# Introduction and motivation

Fluid dynamics



Time-varying dynamical system data is everywhere across multiple scientific and engineering disciplines!

Aeroelasticity

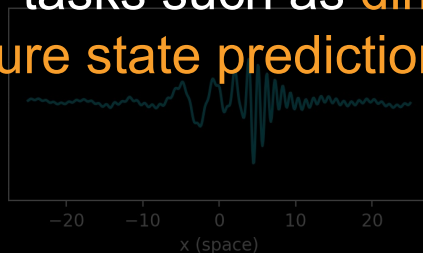


Time-series analysis



How might we leverage data in order to craft models for tasks such as

- reduction, future state prediction, and control?

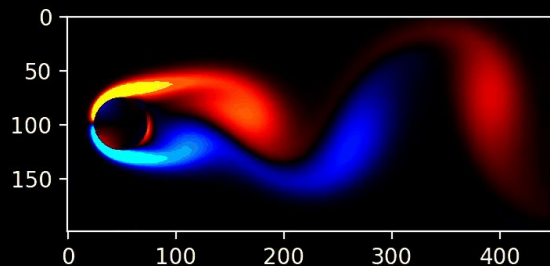


Climate science

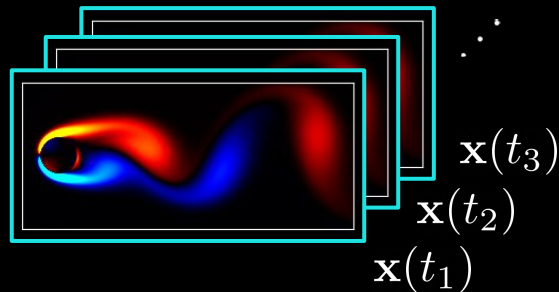


# Learning dynamics from data

Collect data



Each snapshot is an observation of the **state**



$$\underline{\mathbf{x}(t) \in \mathbb{R}^n}$$

Find a **system of equations** that best-describes the observed dynamics

$$\dot{\mathbf{x}}(t) = \underline{f(\mathbf{x}(t))}$$

Method 1: Dynamic Mode Decomposition (DMD)

# Dynamic mode decomposition (DMD)

The **simplest** model that we can find is a **linear** one.

$$\underline{\mathbf{A}} \in \mathbb{R}^{n \times n}$$

$$\dot{\mathbf{x}}(t) = \underline{\mathbf{A}}\mathbf{x}(t)$$

The solution to this general system of equations is known:

$$\mathbf{x}(t) = \begin{bmatrix} | & & | \\ \phi_1 & \dots & \phi_r \\ | & & | \end{bmatrix} \begin{bmatrix} b_1 & & \\ & \ddots & \\ & & b_r \end{bmatrix} \begin{bmatrix} e^{\omega_1 t} \\ \vdots \\ e^{\omega_r t} \end{bmatrix} = \Phi \text{diag}(\mathbf{b}) e^{\omega t}$$

Eigenvectors  
of AAmplitudes for  
reconstructionEigen-  
values  
of A

Schmid, *JFM* 2010.

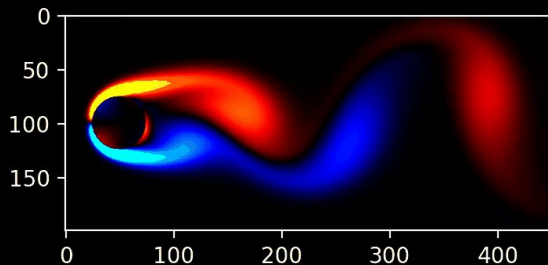
Rowley, Mezic, Bagheri, Schlatter, Henningson, *JFM* 2009.

Tu, Rowley, Luchtenburg, Brunton, Kutz, *JCD* 2014.

Kutz, Brunton, Brunton, Proctor, *SIAM* 2016.

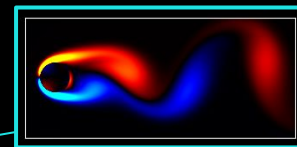
# Dynamic mode decomposition (DMD)

Collect data



Organize data into a matrix

$$\mathbf{X} = \begin{bmatrix} | & | & \dots & | \\ \mathbf{x}(t_1) & \mathbf{x}(t_2) & \dots & \mathbf{x}(t_m) \\ | & | & \dots & | \end{bmatrix}$$



Snapshot at  
time  $t_m$

Decompose into...

Spatial Modes

Amplitudes

Time Dynamics

$$\mathbf{X} \approx \begin{bmatrix} | & & | \\ \phi_1 & \dots & \phi_r \\ | & & | \end{bmatrix} \begin{bmatrix} b_1 & & \\ & \ddots & \\ & & b_r \end{bmatrix} \begin{bmatrix} e^{\omega_1 t_1} & \dots & e^{\omega_1 t_m} \\ \vdots & \ddots & \vdots \\ e^{\omega_r t_1} & \dots & e^{\omega_r t_m} \end{bmatrix} = \Phi \text{diag}(\mathbf{b}) \mathbf{T}(\omega)$$

Schmid, *JFM* 2010.

Rowley, Mezic, Bagheri, Schlatter, Henningson, *JFM* 2009.

Tu, Rowley, Luchtenburg, Brunton, Kutz, *JCD* 2014.

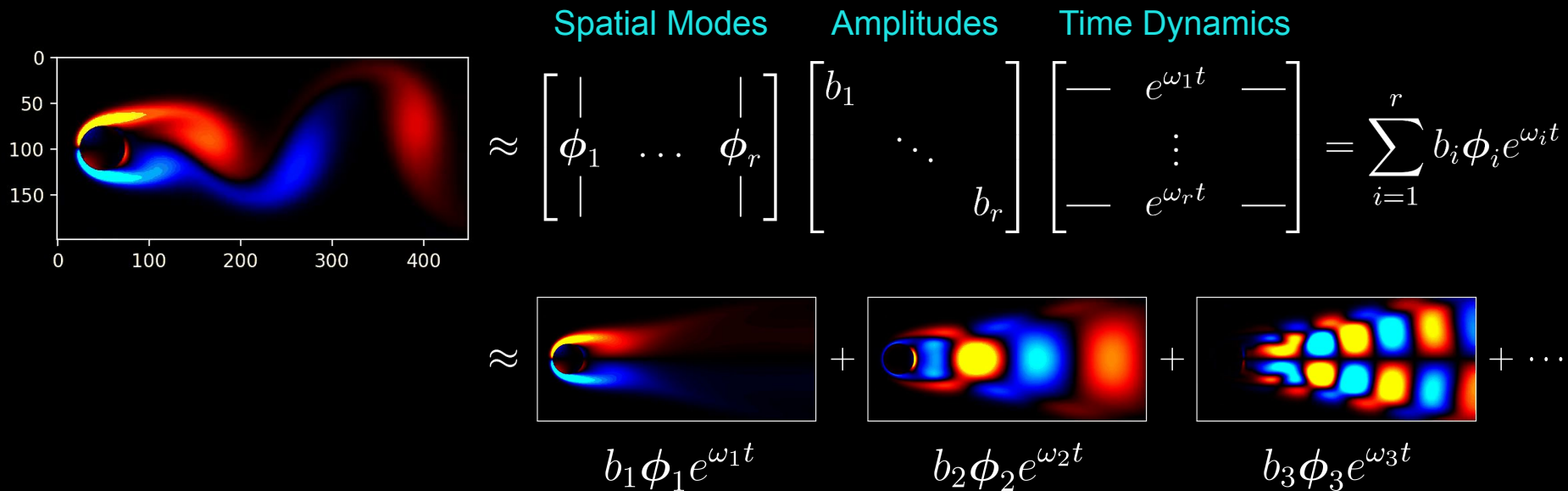
Kutz, Brunton, Brunton, Proctor, *SIAM* 2016.

Download the slides and code and follow along!

[github.com/sichinaga/python-dynamics-tutorial](https://github.com/sichinaga/python-dynamics-tutorial)



# Dynamic mode decomposition (DMD)



Schmid, *JFM* 2010.

Rowley, Mezic, Bagheri, Schlatter, Henningson, *JFM* 2009.

Tu, Rowley, Luchtenburg, Brunton, Kutz, *JCD* 2014.

Kutz, Brunton, Brunton, Proctor, *SIAM* 2016.

# Optimized DMD

Optimization problem:

$$\operatorname{argmin}_{\Phi_b, \omega} \frac{1}{2} \|\mathbf{X} - \Phi_b \mathbf{T}(\omega)\|_F^2$$

Use variable projection for nonlinear least-squares problems with alternating updates:

- **Update the modes:** for a fixed set of DMD eigenvalues, compute

$$\hat{\Phi}_b = \mathbf{X} [\mathbf{T}(\omega)]^\dagger$$

- **Update the eigenvalues:** for a fixed set of DMD modes, compute a local optimizer

$$\hat{\omega} = \operatorname{argmin}_{\omega} \frac{1}{2} \|\mathbf{X} - \hat{\Phi}_b [\mathbf{T}(\omega)]^\dagger \mathbf{T}(\omega)\|_F^2$$

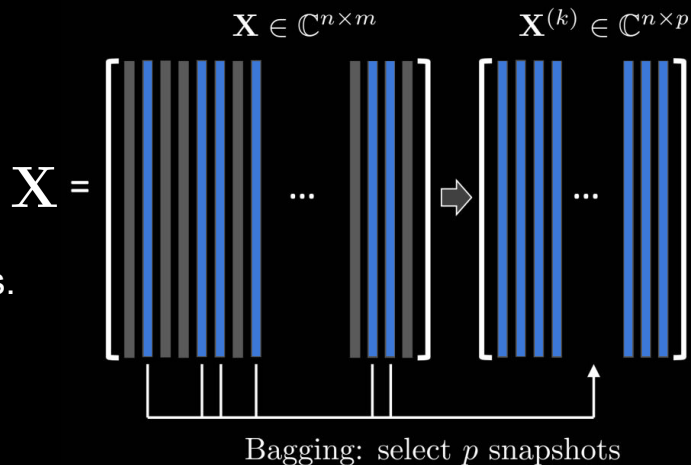
via methods such as Levenberg-Marquardt.

# Optimized DMD with bagging (BOP-DMD)

$$\arg\min_{\Phi, \omega} \frac{1}{2} \|\mathbf{X} - \Phi \mathbf{b} \mathbf{T}(\omega)\|_F^2$$

- Optimally suppresses bias from noise.
- Handles snapshots that are unevenly sampled in time.
- Allows for regularization and constraints.
- **Requires solving a nonlinear optimization problem.**
  - **Stabilize with bagging.**
  - Gives UQ metrics.

## Optimized DMD with bagging

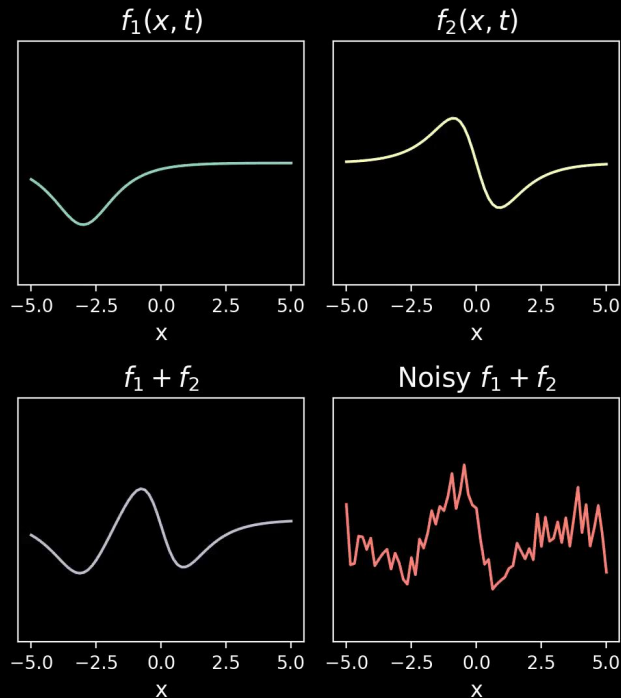


Optimized DMD  
 $\Rightarrow \mathbf{x}^{(k)} = \Phi^{(k)} \exp(\Omega^{(k)} t) \mathbf{b}^{(k)}$   
 $k = 1, 2, \dots, K$   
Compute after  $K$  trials  
Mean:  $\Phi, \Omega, \mathbf{b}$   
Variance:  $\Phi, \Omega, \mathbf{b}$

# PyDMD: A Python package for DMD



$t = 12.56637$

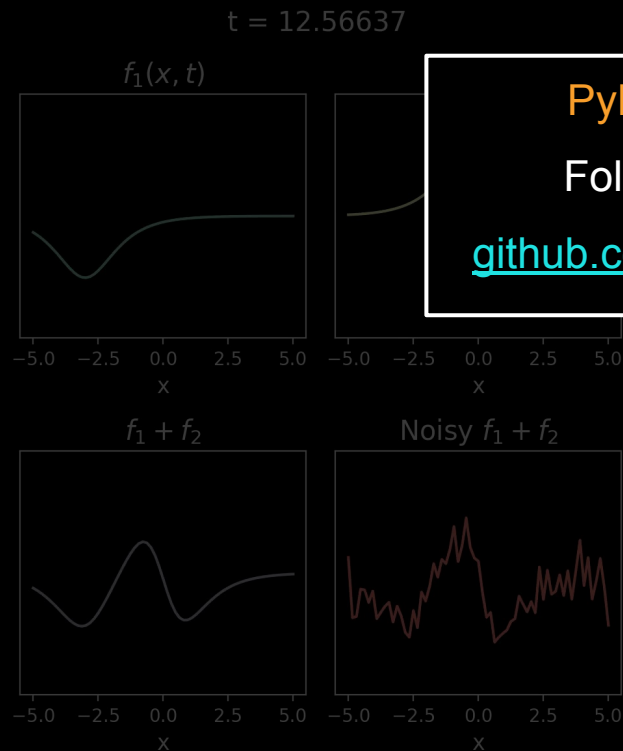


```
1 from pydmd import BOPDMD
2 from pydmd.preprocessing import hankel_preprocessing
3 from pydmd.plotter import plot_summary
4
5 bopdmd = BOPDMD(
6     svd_rank=4,
7     num_trials=0,
8     eig_constraints={"conjugate_pairs"},
9 )
10 delay_bopdmd = hankel_preprocessing(bopdmd, d=2)
11 delay_bopdmd.fit(X, t=t[:-1])
12 plot_summary(delay_bopdmd, x=x, t=t[:-1], d=2)
13
14
```

MADE WITH GIPHY



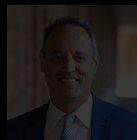
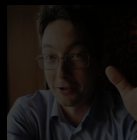
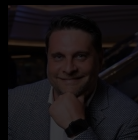
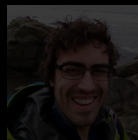
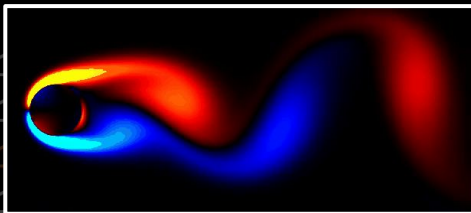
# PyDMD: A Python package for DMD



## PyDMD Demo: Flow Past a Cylinder

Follow along using the code found at

[github.com/sichinaga/python-dynamics-tutorial](https://github.com/sichinaga/python-dynamics-tutorial)



Download the slides and code and follow along!  
[github.com/sichinaga/python-dynamics-tutorial](https://github.com/sichinaga/python-dynamics-tutorial)

Demo, Tezzele, Rozza, *JOSS* 2018.  
Ichinaga, Andreuzzi, Demo, Tezzele, Lapo, Rozza, Brunton, Kutz, *JMLR* 2024.

Method 2: Sparse Identification of Nonlinear Dynamics (SINDy)

# Sparse Identification of Nonlinear Dynamics (SINDy)

Instead of a **linear** model (DMD)...

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t)$$

$$\dot{x}_1(t) = \underline{a}x_1(t) + \underline{b}x_2(t) + \cdots + \underline{c}x_n(t)$$

$\vdots$

Suppose we instead looked for a **nonlinear** model (SINDy),  
which permits the use of nonlinear terms such as:

$$ax_1^2(t)$$

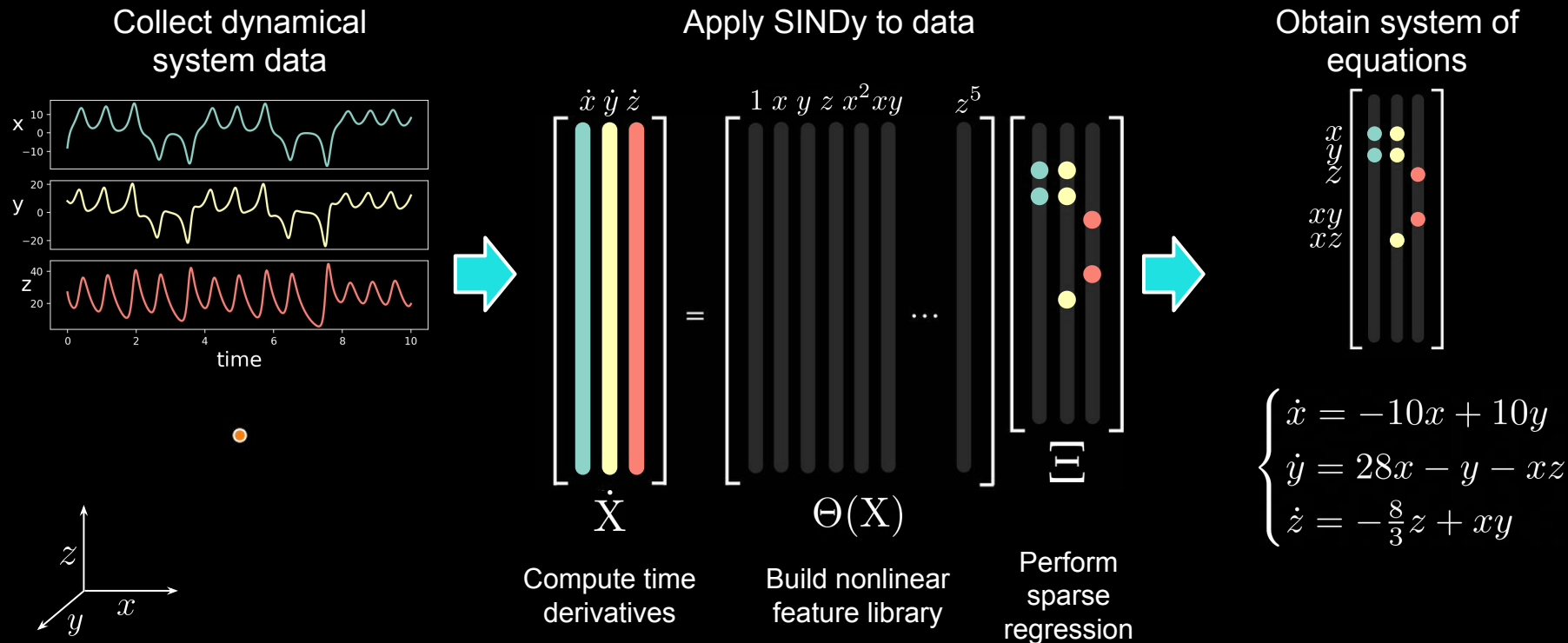
Higher-order polynomials

$$bx_1(t)x_2(t)$$

$$c \sin(x_1(t))$$

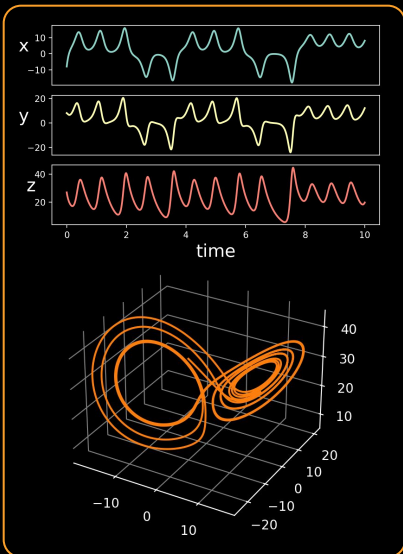
Nonlinear functions

# Sparse Identification of Nonlinear Dynamics (SINDy)





# PySINDy: A Python package for SINDy



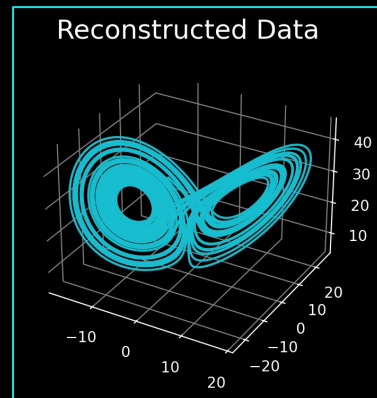
```
import pysindy as ps

differentiation_method = ps.FiniteDifference(order=2)
feature_library = ps.PolynomialLibrary(degree=3)
optimizer = ps.STLSQ(threshold=0.2)

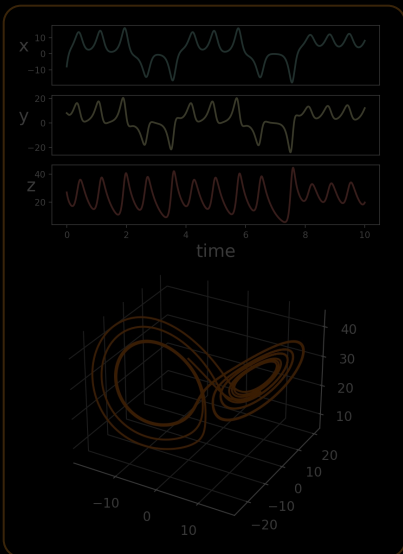
model = ps.SINDy(
    differentiation_method=differentiation_method,
    feature_library=feature_library,
    optimizer=optimizer,
    feature_names=["x", "y", "z"],
)

model.fit(X, t=t)

model.print()
X_reconstruction = model.simulate(x0=X[0], t=t_long)
```


$$\begin{aligned}(\mathbf{x})' &= -9.999 \mathbf{x} + 9.999 \mathbf{y} \\ (\mathbf{y})' &= 27.992 \mathbf{x} + -0.999 \mathbf{y} + -1.000 \mathbf{x} \mathbf{z} \\ (\mathbf{z})' &= -2.666 \mathbf{z} + 1.000 \mathbf{x} \mathbf{y}\end{aligned}$$

# PySINDy: A Python package for SINDy



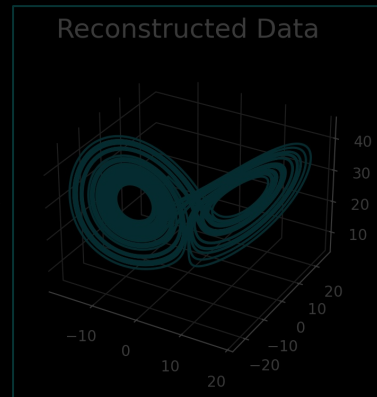
```
import pysindy as ps
```

## PySINDy Demo: Predator-Prey System

Follow along using the code found at

[github.com/sichinaga/python-dynamics-tutorial](https://github.com/sichinaga/python-dynamics-tutorial)

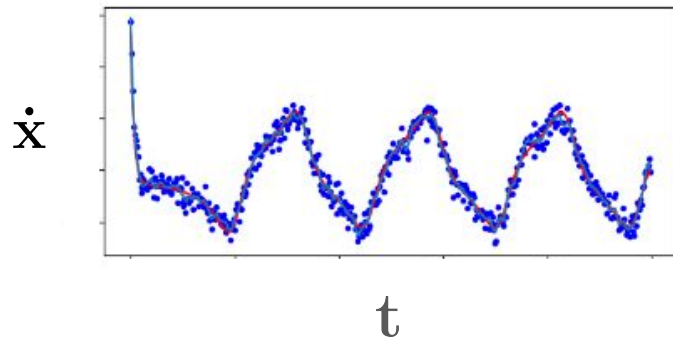
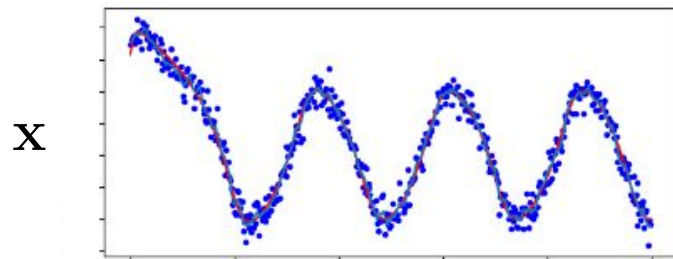
```
optimizer=optimizer,  
feature_names=["x", "y", "z"],  
)  
model.fit(X, t=t)  
  
model.print()  
X_reconstruction = model.simulate(x0=X[0], t=t_long)
```



$$\begin{aligned}(x)' &= -9.999 x + 9.999 y \\(y)' &= 27.992 x + -0.999 y + -1.000 x z \\(z)' &= -2.666 z + 1.000 x y\end{aligned}$$

# Does SINDy work? Van der Pol Oscillator

$$\ddot{x} = -\mu x^2 \dot{x} + \mu y - x$$



Coefficients of ODE

True

|              | $\dot{x}$ | $\ddot{x}$ |
|--------------|-----------|------------|
| 1            |           |            |
| $x$          |           |            |
| $\dot{x}$    |           |            |
| $x^2$        |           |            |
| $x\dot{x}$   |           |            |
| $\dot{x}^2$  |           |            |
| $x^3$        |           |            |
| $x^2\dot{x}$ |           |            |
| $x\dot{x}^2$ |           |            |
| $\dot{x}^3$  |           |            |

Color  
Scale:

++

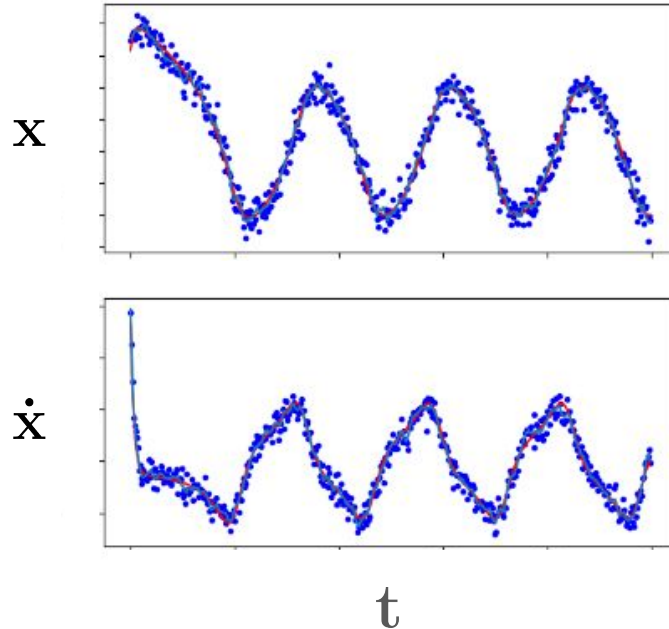
+

-

--

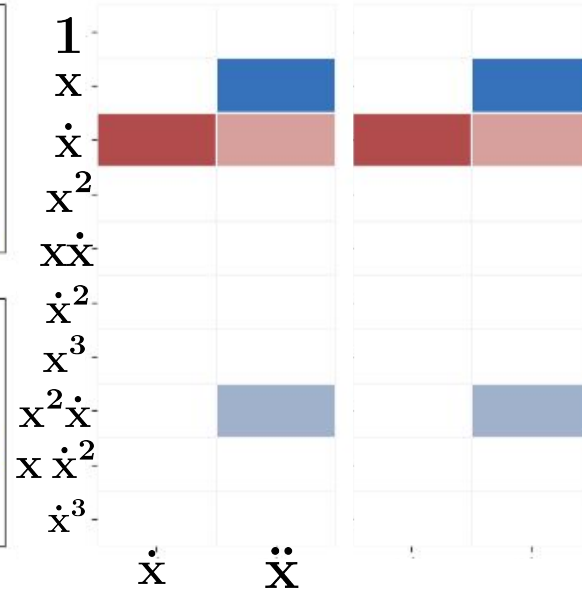
# Does SINDy work? Van der Pol Oscillator

$$\ddot{x} = -\mu x^2 \dot{x} + \mu y - x$$



Coefficients of ODE

True



Color Scale:

++

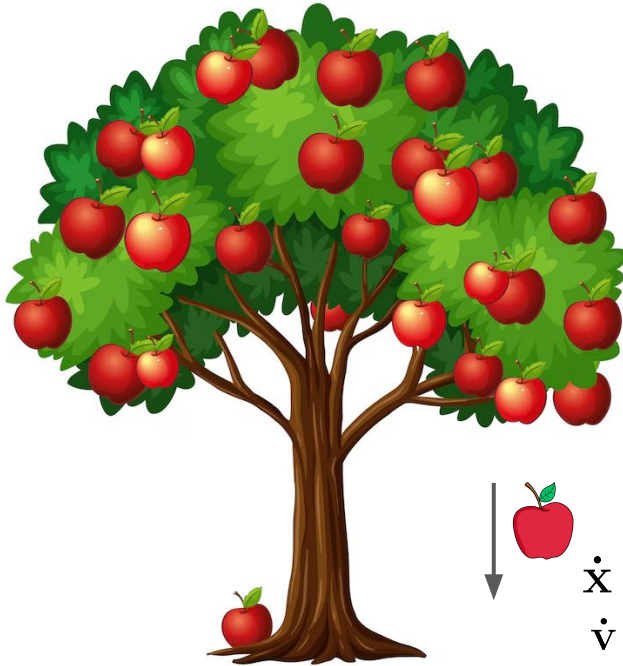
+

-

--

# Does SINDy always work?

Depiction of system



$$\dot{x} = v$$
$$\dot{v} = g$$

Coefficients of ODE  
True

|       |           |           |
|-------|-----------|-----------|
| 1     |           |           |
| $x$   |           |           |
| $v$   |           |           |
| $x^2$ |           |           |
| $xv$  |           |           |
| $v^2$ |           |           |
|       | $\dot{x}$ | $\dot{v}$ |

Color  
Scale:

++

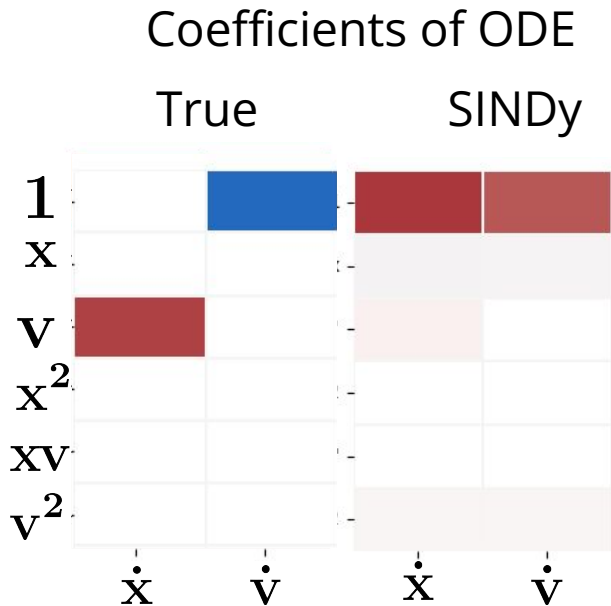
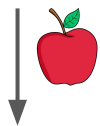
+

-

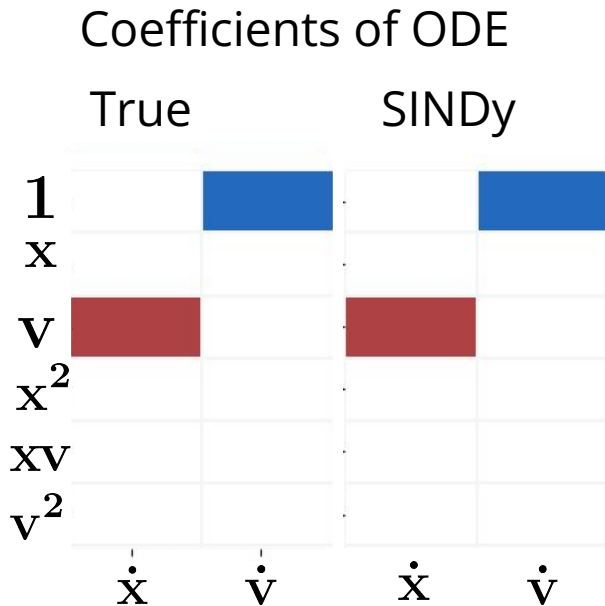
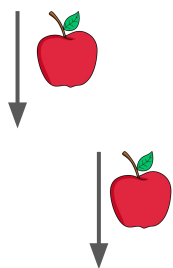
--

# Nonlinear (and sparse) is challenging!

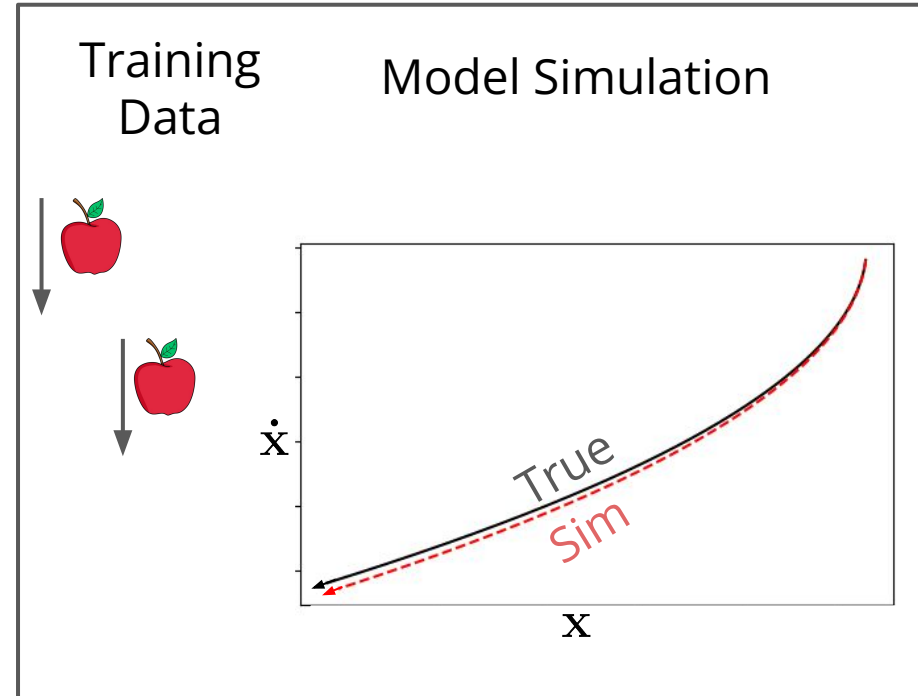
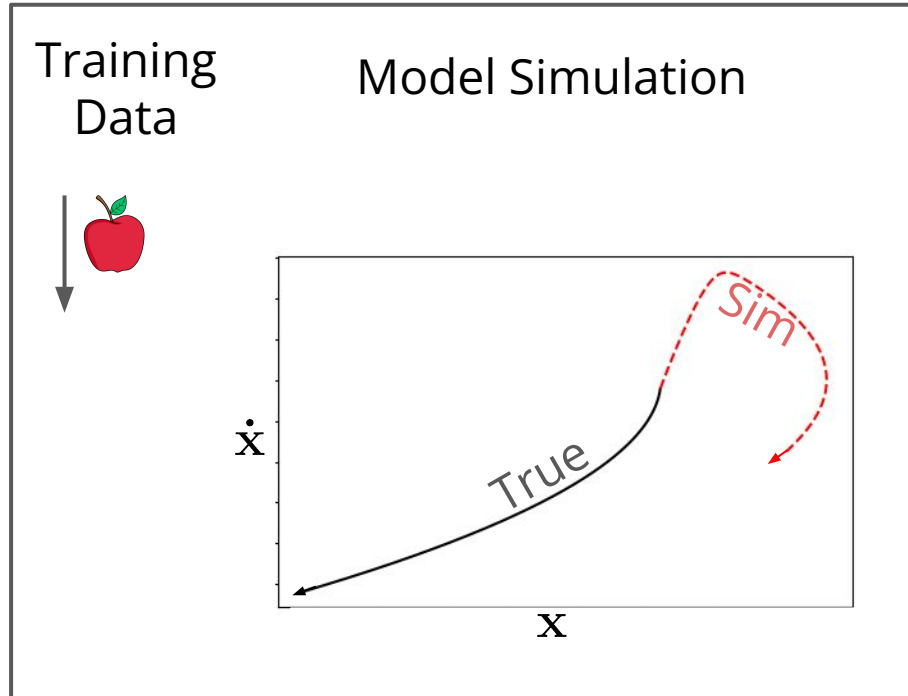
Training  
Data



Training  
Data



# Nonlinear (and sparse) is challenging!



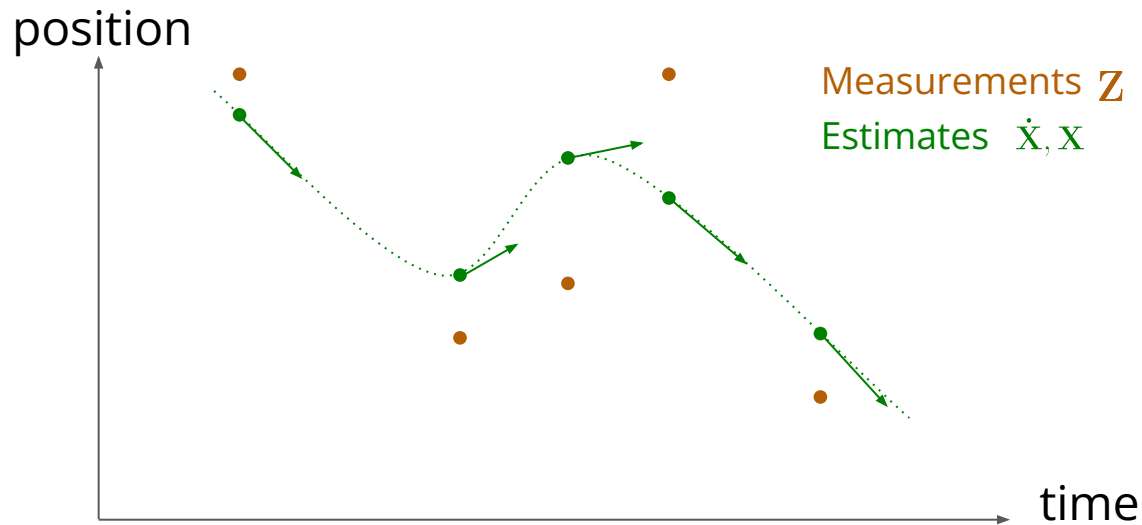
# Step details



0. Measure some data

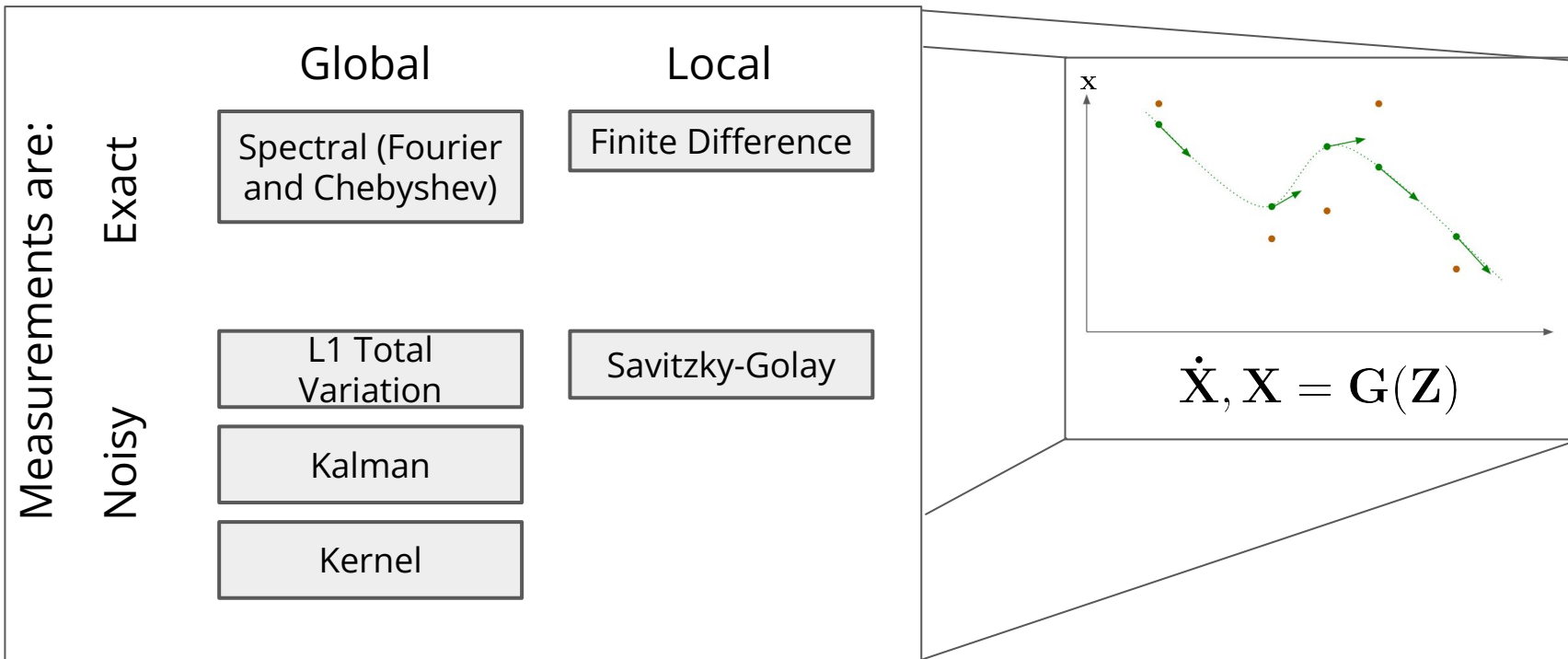


# Differentiation step

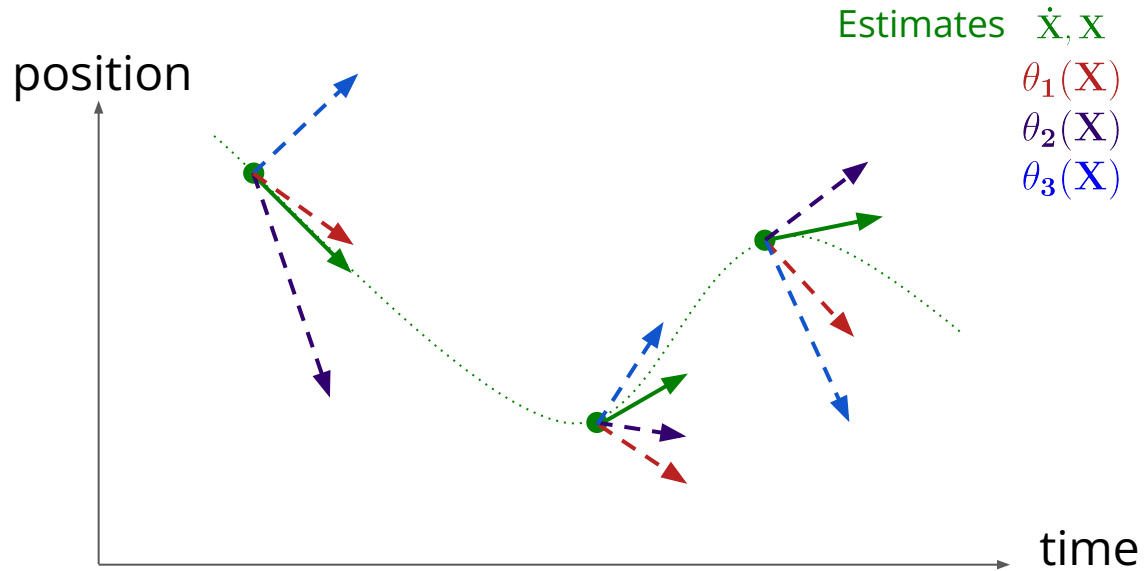


1. Choose a Smoother,  $G$ , and estimate  $\dot{X}, X = G(Z)$

# Differentiation options

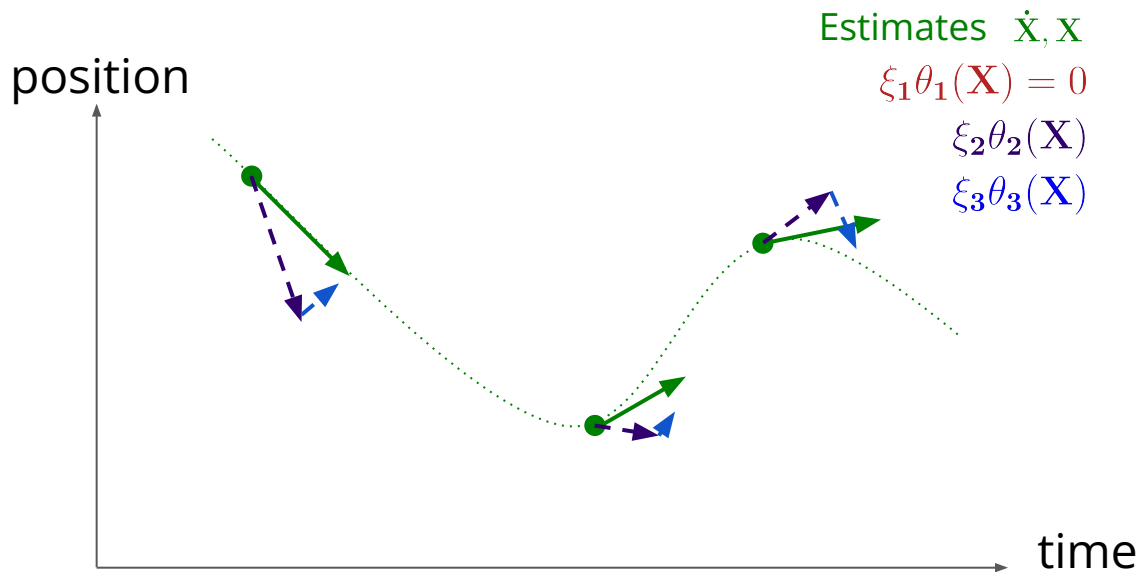


# Feature selection step



2. Choose a function library  $\Theta = [\theta_1 \dots, \theta_n]$  and evaluate  $\Theta(\mathbf{X})$

# Sparse method step:



3. Choose a sparse regression method and solve  $\arg \min_{\text{sparse } \Xi} \|\dot{\mathbf{X}} - \Xi^T \Theta(\mathbf{X})\|$

# Sparse method options

unconstrained

STLSQ  
(Sequential Threshold  
Least Squares)

SSR  
(Stepwise Sparse  
Regression)

FROLS  
(Forward Regression,  
Ordinary Least  
Squares)

Regularized  
Spike-Slab prior

Constraints

SR3  
(Sparse Relaxed  
Regularized  
Regression)

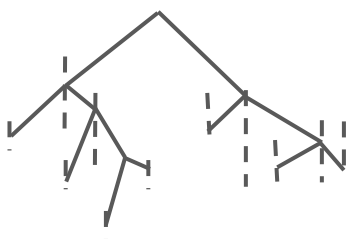
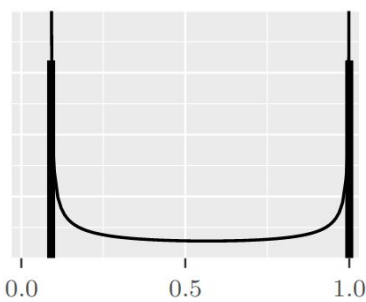
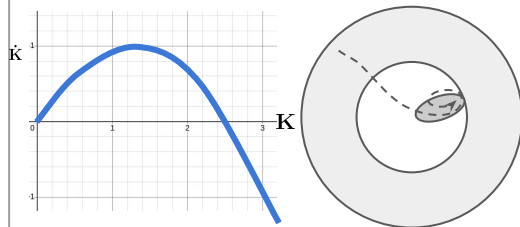
MIOSR  
(Mixed Integer  
Optimization, Sparse  
Regression)

Trapping  
(An SR3 variant)

Ensemble to get statistics

Monte-Carlo to get  
statistics

# New Sparse methods

|                  |                                                                                                   |                                                                                    |                                                                                                                       |
|------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
|                  | Mixed Integer Optimization,<br>sparse regression                                                  | Regularized Horseshoe Prior,<br>Monte Carlo Estimation                             | Trapping (extended)                                                                                                   |
| Authors          | Gurnee & Bertsimas [2022]                                                                         | Hirsh et al [2021]                                                                 | Peng et al [2024]                                                                                                     |
| Use              | Known system benchmark                                                                            | Inclusion probabilities                                                            | Bounded trajectories                                                                                                  |
| Backup<br>Slides | $\ \Xi\ _0 < k$  |  |  $Q_{ijk} + Q_{jki} + Q_{kij} = 0$ |

```
pip install git+https://github.com/dynamicslab/pysindy@master
```

# Conclusion and Method Overview

## Dynamic Mode Decomposition (DMD)

### Pros:

- Models are **simple** and linear.
- Breaks data down into interpretable set of **spatiotemporal components**.
- Fast, noise-robust algorithms and method variants are available.

### Cons:

- **Cannot** model certain **complex nonlinear** behaviors.
- Sometimes requires the use of **many spatiotemporal modes**.

## Sparse Identification of Nonlinear Dynamics (SINDy)

### Pros:

- Models are nonlinear and can describe **complex nonlinear dynamics**.
- Models are **concise** and **readable** thanks to sparsity.
- Also has variants and fast algorithms.

### Cons:

- Sparse regression on dictionary matrix can be very **large** and **costly**.
- Data must be **well-resolved** enough to accurately compute derivatives.



# Resources and Further Reading

## DMD:

- DMD Book: <https://epubs.siam.org/doi/book/10.1137/1.9781611974508>
- Optimized DMD: <https://epubs.siam.org/doi/10.1137/M1124176>
- PyDMD package new paper (long ver): <https://arxiv.org/abs/2402.07463>
- PyDMD package new paper (short ver): <https://www.jmlr.org/papers/v25/24-0739.html>
- CODE: <https://github.com/PyDMD/PyDMD>

## SINDy:

- Original SINDy paper: <https://www.pnas.org/doi/10.1073/pnas.1517384113>
- PySINDy package paper 1: <https://arxiv.org/abs/2004.08424>
- PySINDy package paper 2: <https://arxiv.org/abs/2111.08481>
- CODE: <https://github.com/dynamicslab/pysindy>

# References

- P. J. Schmid, *Dynamic mode decomposition of numerical and experimental data*, Journal of Fluid Mechanics, 656 (2010), pp. 5–28.
- C. W. Rowley, I. Mezic, S. Bagheri, P. Schlatter, and D. S. Henningson, *Spectral analysis of nonlinear flows*, Journal of Fluid Mechanics, 641 (2009), pp. 115–127.
- J. H. Tu, C. W. Rowley, D. M. Luchtenburg, S. L. Brunton, and J. N. Kutz, *On dynamic mode decomposition: Theory and applications*, Journal of Computational Dynamics, 1 (2014), pp. 391–421.
- J. N. Kutz, S. L. Brunton, B. W. Brunton, and J. L. Proctor, *Dynamic Mode Decomposition: Data-Driven Modeling of Complex Systems*, Society for Industrial and Applied Mathematics, Philadelphia, PA, 2016.
- T. Askham and J. N. Kutz, *Variable projection methods for an optimized dynamic mode decomposition*, SIAM Journal on Applied Dynamical Systems, 17 (2018), pp. 380–416.
- D. Sashidhar and J. N. Kutz, *Bagging, optimized dynamic mode decomposition for robust, stable forecasting with spatial and temporal uncertainty quantification*, Proceedings of the Royal Society A, 380 (2022), p. 20210199.
- N. A. Rayner, D. E. Parker, E. B. Horton, C. K. Folland, L. V. Alexander, D. P. Rowell, E. C. Kent, and A. Kaplan, *Global analyses of sea surface temperature, sea ice, and night marine air temperature since the late nineteenth century*, Journal of Geophysical Research: Atmospheres, 108 (2003).

# References

- N. Demo, M. Tezzele, and G. Rozza, *PyDMD: Python dynamic mode decomposition*, Journal of Open Source Software, 3 (2018), p. 530.
- S. M. Ichinaga, F. Andreuzzi, N. Demo, M. Tezzele, K. Lapo, G. Rozza, S. L. Brunton, and J. N. Kutz, *PyDMD: a Python package for robust dynamic mode decomposition*, JMLR, 25 (2024), pp. 1–9.
- S. L. Brunton, J. L. Proctor, and J. N. Kutz, *Discovering governing equations from data by sparse identification of nonlinear dynamical systems*, Proceedings of the National Academy of Sciences, 113 (2016), pp. 3932–3937.
- B. M. de Silva, K. Champion, M. Quade, J. Loiseau, J. N. Kutz, and S. L. Brunton, *PySINDy: A Python package for the sparse identification of nonlinear dynamical systems from data*, Journal of Open Source Software, 5 (2020), p. 2104.
- A. A. Kaptanoglu, B. M. de Silva, U. Fasel, K. Kaheman, A. J. Goldschmidt, J. Callahan, C. B. Delahunt, Z. G. Nicolaou, K. Champion, J. Loiseau, J. N. Kutz, and S. L. Brunton, *PySINDy: A comprehensive Python package for robust sparse system identification*, Journal of Open Source Software, 7 (2022), p. 3994.